



JIMMA INSTITUTE OF TECHNOLOGY

FACULTY OF COMPUTING AND INFORMATICS

DPARTMENT OF SOFTWARE ENGINEERING

NEWS WEB APPLICATION

Group Member's Name	ID No
Abenezer Tamiru	Ru 2154/14
Betselot Abraham	Ru1612/14
Mengistu Tadesse	Ru1187/14
Natnael Bayu	Ru1408/14
Eman Hussen	Ru0279/14

Table of Contents

1. Introduction	1
2. System Architecture Overview	1
2.1 Client Layer (Frontend)	3
2.2 Server Layer (Backend – Node.js/Express)	3
2.3 External Services (APIs).....	6
3. Detailed Implementation Steps	6
3.1 Backend Development.....	6
3.1.1 Environment Setup	6
3.1.2 Creating the Express Server.....	7
3.1.3 User Authentication (JWT).....	7
User Data (Mock Database).....	7
Login Route	7
3.1.4 Authentication Middleware	8
3.1.5 Logging Middleware (Winston)	8
3.1.6 Categories Route.....	9
3.1.7 News API Integration	9
Primary Source: The Guardian API.....	9
Data Transformation	9
3.1.8 Fallback NewsAPI Integration.....	10
3.2 Frontend Development	10
3.2.1 Login Page	10
3.2.2 Dashboard Page	10
3.2.3 Fetching News	11
3.2.4 Displaying News Cards	11
3.2.5 Pagination	11
4. Tools and Technologies Used.....	11

Programming Languages	11
Backend Tools	12
Frontend Tools.....	12
External APIs.....	12
Development Tools.....	12
5. System Testing	12
5.1 Unit Testing	12
5.2 Integration Testing.....	13
5.3 UI Testing.....	13
5.4 Error Testing.....	13
6. Results and Observations.....	14
6.1 Successful System Behavior	14
7. Challenges	14
Guardian image availability issues	14
Rate limiting	14
CORS issues	14
Token expiration.....	14
8. Conclusion	14
9. References	16

Figures

Figure 1: Login Page.....	1
Figure 2:Home Page	2
Figure 3:Fetches News From API	2
Figure 4:Login Posman Result	4
Figure 5: Fetched News	4
Figure 6:Search News	5
Figure 7:Get news Category	5

1. Introduction

In today's digital era, accessing timely, reliable, and categorized news is essential for informed decision-making. The rise of web applications powered by APIs has enabled the creation of platforms that aggregate news from multiple sources and deliver up-to-date information instantly. This project—News Web Application—was developed to demonstrate the integration of secure backend services, external APIs, authentication mechanisms, and dynamic frontend interfaces.

The system provides users with access to real-time news articles fetched from trusted sources such as The Guardian API and NewsAPI.org. To ensure security and access control, the application uses JSON Web Tokens (JWT) for user authentication. The backend is built using Node.js and Express, while the frontend uses HTML, CSS, and JavaScript, offering a responsive and interactive user experience.

This report provides a comprehensive explanation of the entire development process, covering system architecture, code implementation, tools used, results, observations, testing procedures, and the overall conclusion.

2. System Architecture Overview

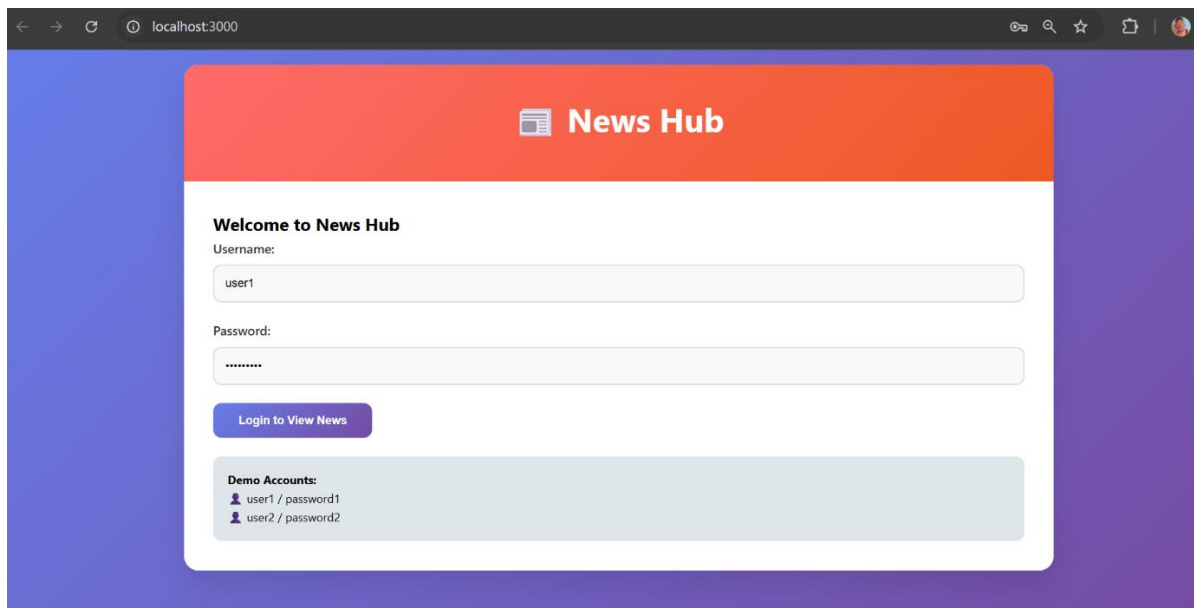


FIGURE 1: LOGIN PAGE

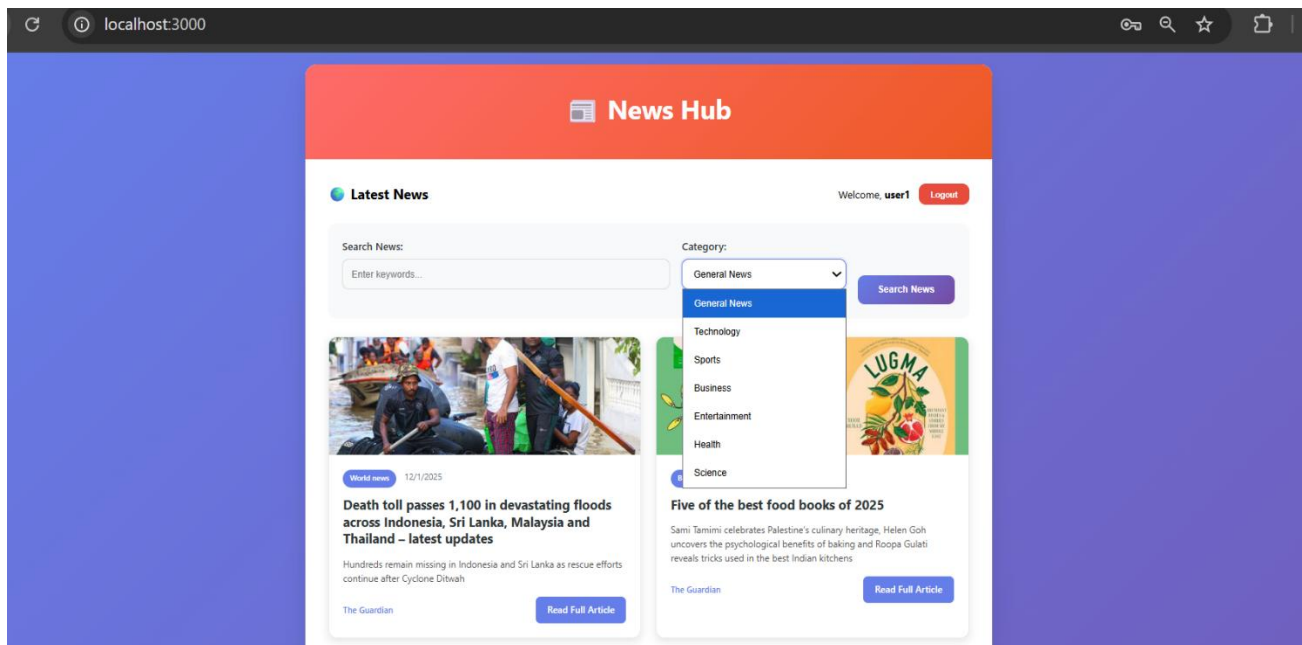


FIGURE 2:HOME PAGE

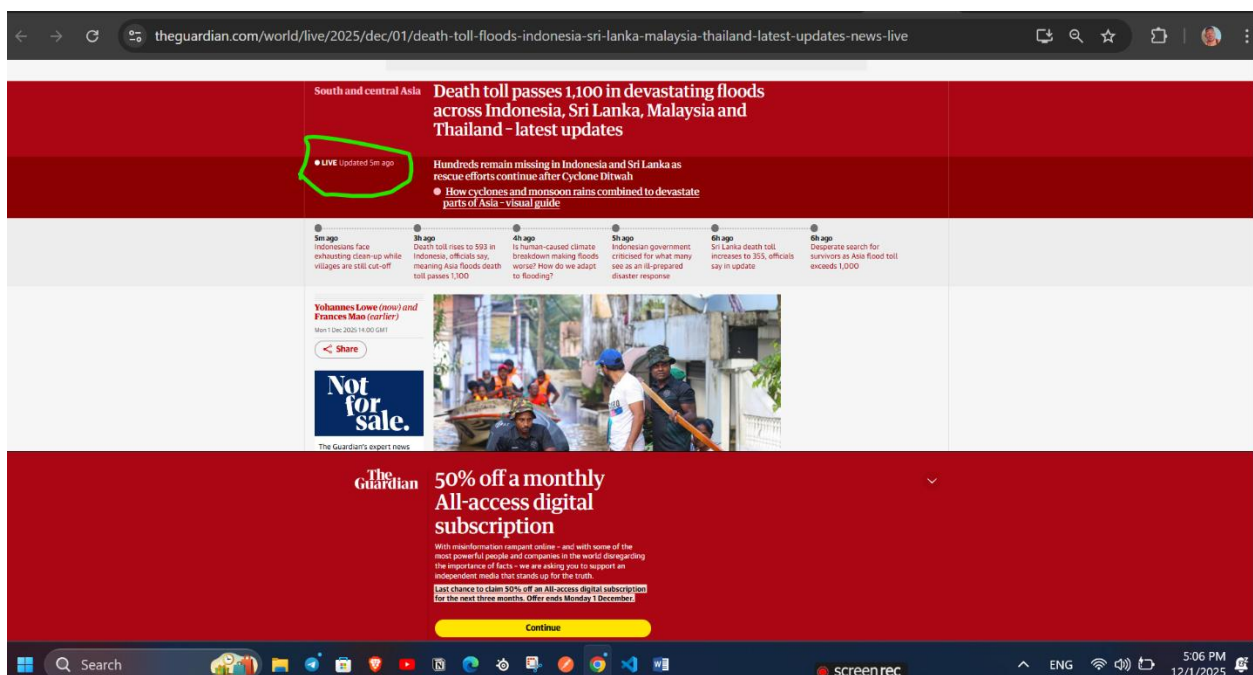


FIGURE 3:FETCHED NEWS FROM API

2.1 Client Layer (Frontend)

- Built using HTML, CSS, JavaScript
- Handles:
 - user interface
 - login form
 - category selection
 - search functionality
 - pagination
 - display of news cards
 - token management via localStorage

2.2 Server Layer (Backend – Node.js/Express)

- REST API endpoints
- Authentication using JWT
- Routes:
 - /api/login
 - /api/news
 - /api/categories
- Middleware:
 - CORS
 - JSON parsing
 - Authentication middleware
 - Logging middleware (Winston)
 - Error-handling middleware
- Fetches data from external APIs (Guardian + fallback)

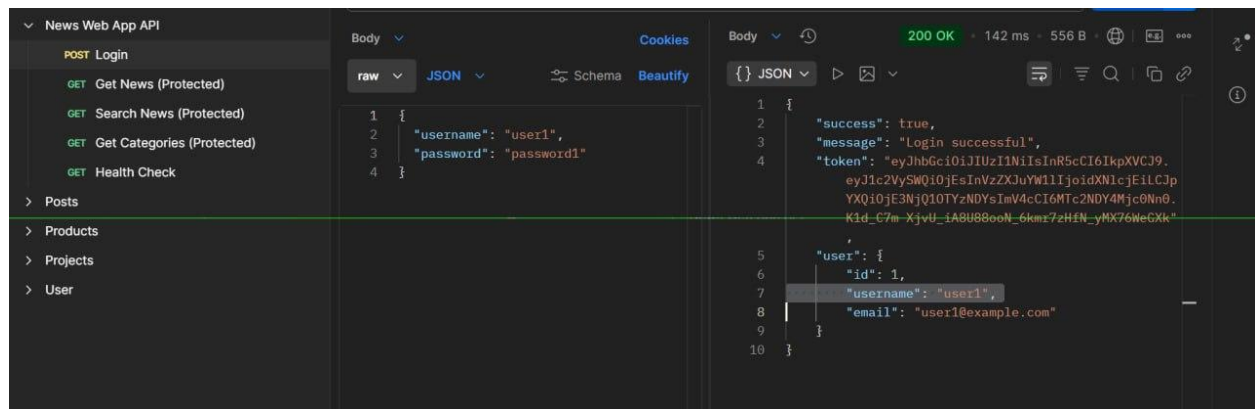


FIGURE 4: LOGIN POSMAN RESULT

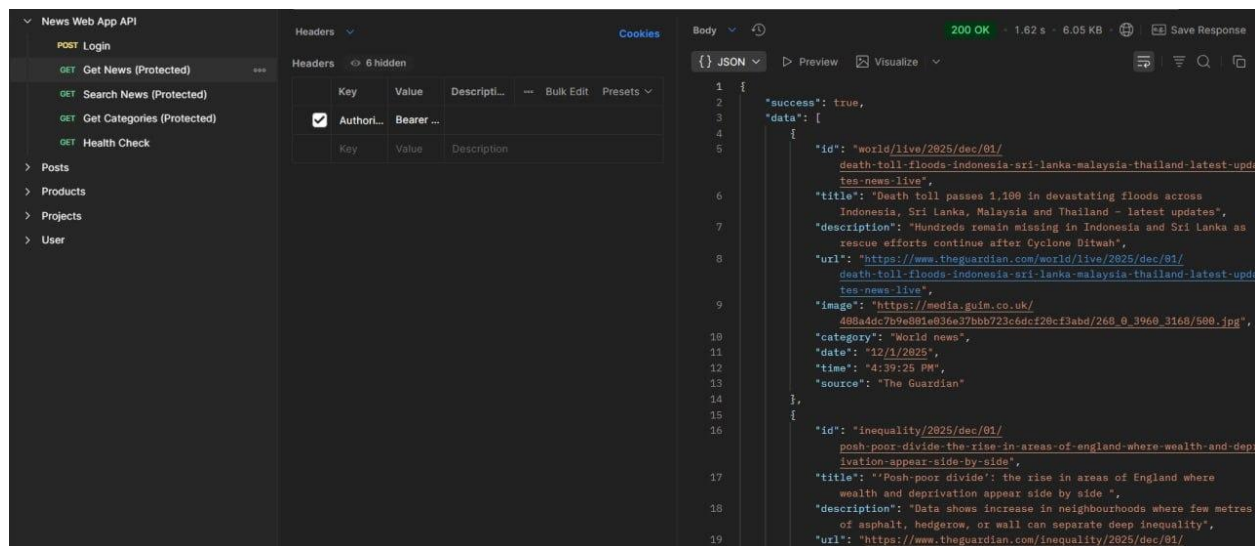


FIGURE 5: FETCHED NEWS

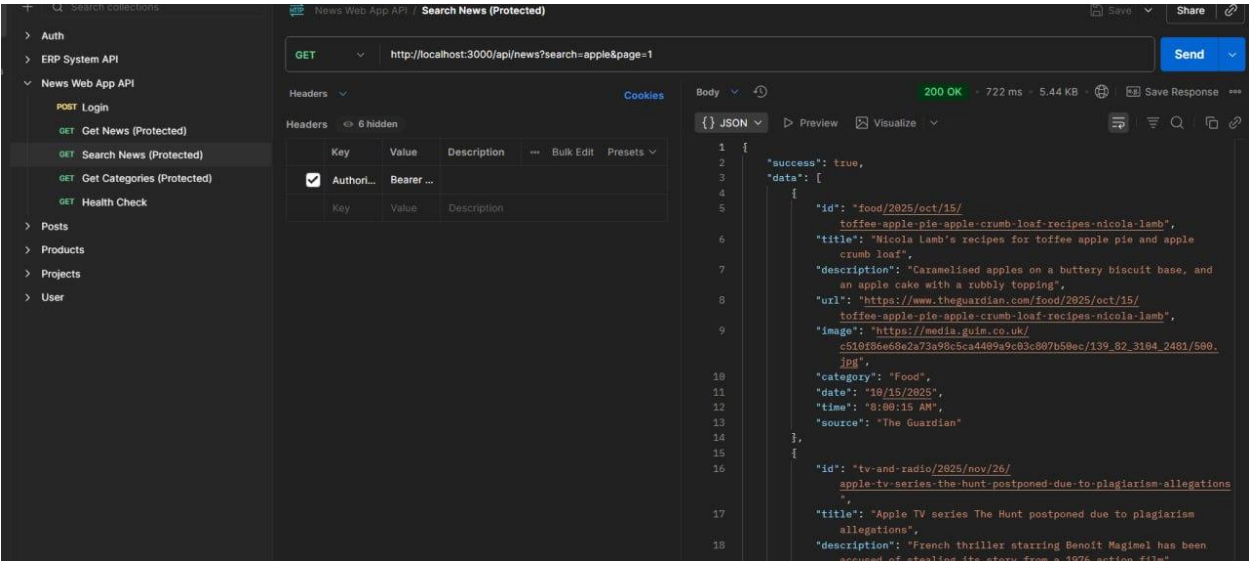


FIGURE 6:SEARCH NEWS

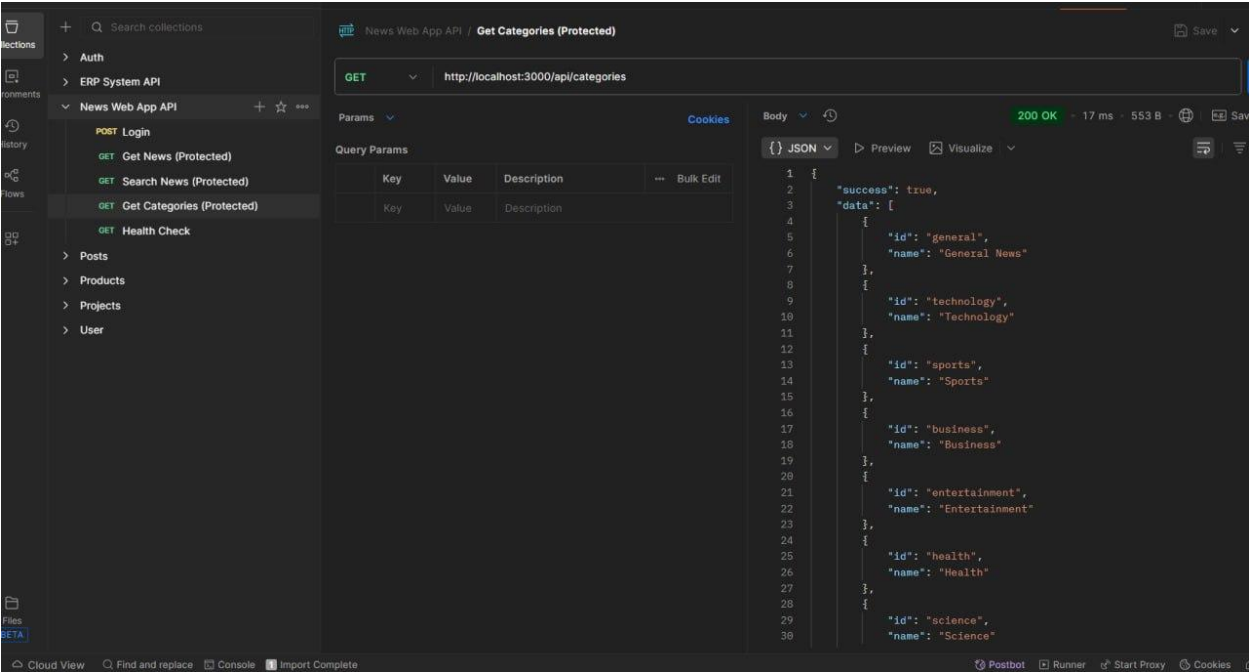


FIGURE 7:GET NEWS CATEGORY

2.3 External Services (APIs)

- The Guardian Open Platform
- NewsAPI.org (fallback)
- Planned: Weather API (module exists, integration future)

3. Detailed Implementation Steps

3.1 Backend Development

3.1.1 Environment Setup

Tools Installed:

- Node.js (v18+)
- NPM/Yarn
- Express.js
- Axios
- jsonwebtoken
- winston
- cors
- dotenv

Project Structure:

/project

 /middleware

 auth.js

 logger.js

 /config

 api.config.js

 /public

 News_app_api_report

 index.html

 app.js

```
styles.css
server.js
package.json
```

3.1.2 Creating the Express Server

The backend server is built using Express. Key configurations include:

- CORS enabled for cross-origin requests
- JSON request body parsing
- Request logging
- Static file serving for frontend

Sample:

```
const express = require("express");
const app = express();
app.use(express.json());
app.use(cors());
app.use(express.static("public"));
```

3.1.3 User Authentication (JWT)

User Data (Mock Database)

```
const users = [
  { id: 1, username: "user1", password: "password1", email: "user1@example.com" },
  { id: 2, username: "admin", password: "admin123", email: "admin@example.com" }
];
```

Login Route

Steps:

1. Validate username + password.
2. Generate JWT with:
 - expiry: 24 hours
 - encrypted payload

3. Return token to client.

Sample:

```
const token = jwt.sign({ id: user.id, username: user.username }, "secretKey", { expiresIn: "24h" });
```

3.1.4 Authentication Middleware

The middleware ensures that only authorized users can access /api/news.

Functions:

- Verifies JWT from the Authorization header
- Extracts token
- Validates signature
- Handles expired or invalid tokens
- Passes control to next handler

Error responses:

- 401 Unauthorized
- 403 Forbidden

3.1.5 Logging Middleware (Winston)

Winston provides detailed system logs:

- timestamp
- IP address
- request URL
- method
- response time
- status code

Example Log Entry:

info: GET /api/news 200 - 145ms ::1

3.1.6 Categories Route

Returns static category data:

- general
- sports
- technology
- business
- politics
- health
- environment
- world
- education
- entertainment

3.1.7 News API Integration

Primary Source: The Guardian API

- Does not require API key (when using test endpoint)
- Sections map to categories
- Supports:
 - pagination
 - search keywords
 - filtering by category

Data Transformation

Guardian articles return:

- id
- webTitle
- webUrl

- fields.thumbnail
- fields.trailText
- fields.publicationDate

Converted into custom format:

```
{
  id, title, description, image, url, date, source: "The Guardian"
}
```

3.1.8 Fallback NewsAPI Integration

If Guardian fails:

- fallback to newsapi.org
- data mapping similar structure

3.2 Frontend Development

3.2.1 Login Page

Features:

- Form input validation
- Sends POST /api/login
- Saves JWT in localStorage
- Redirects to dashboard

3.2.2 Dashboard Page

Contains:

- Category dropdown
- Search bar
- Logout button
- Dynamic news listing

- Pagination buttons

3.2.3 Fetching News

The frontend attaches token:

```
headers: {  
  Authorization: "Bearer " + localStorage.getItem("token")  
}
```

3.2.4 Displaying News Cards

Each card shows:

- image
- title
- source
- description
- date
- "Read More" link

Fully responsive grid layout.

3.2.5 Pagination

Displays:

- current page
- next/previous buttons

Dynamic based on API metadata.

4. Tools and Technologies Used

Programming Languages

- JavaScript (Node + Browser)

Backend Tools

- Express.js
- JWT (authentication)
- Axios (API consumption)
- Winston (logging)

Frontend Tools

- HTML5
- CSS3 (responsive design)
- Vanilla JavaScript
- Axios CDN

External APIs

- The Guardian Open Platform
- NewsAPI.org

Development Tools

- VS Code
- Postman
- Git + GitHub

5. System Testing

5.1 Unit Testing

Tested modules:

- authentication

- category route
- news fetching
- token validation
- error handling

5.2 Integration Testing

Tools: Postman

Tested:

- login → token → protected routes
- category + pagination
- search + category combination

5.3 UI Testing

Checked:

- responsiveness
- cross-browser compatibility
- layout behavior on mobile/tablet

5.4 Error Testing

- incorrect login
- expired token
- invalid category
- API request failure
- network timeout simulation

6. Results and Observations

6.1 Successful System Behavior

- Login works with secure token issuance.
- Protected routes reject unauthorized access.
- Categories load instantly.
- News articles fetched correctly from Guardian API.
- Fallback works when Guardian fails.
- Frontend fully responsive and interactive.
- Logs stored for debugging and analytics.

7. Challenges

Guardian image availability issues

- Some articles lack images → required fallback logic.

Rate limiting

- NewsAPI free plan limited → fallback must be optimized.

CORS issues

- Had to enable and configure properly.

Token expiration

- Users must re-login after 24 hours.

8. Conclusion

This News Web Application successfully integrates:

- secure user authentication,
- external API communication,
- data transformation,
- error handling,
- logging,
- responsive frontend design.

The project demonstrates strong understanding of:

- REST API development
- token-based security
- API integration and transformation
- JavaScript-based frontend development
- backend–frontend communication

This application can be further expanded to include:

- weather integration,
- user preference storage,
- bookmarks and saved articles,
- admin dashboard,
- multi-language support.

9. References

1. Express.js. (2024). *Express web framework documentation*. <https://expressjs.com>
2. Node.js Foundation. (2024). *Node.js v18 documentation*. <https://nodejs.org>
3. The Guardian. (2024). *Open Platform API documentation*. <https://open-platform.theguardian.com>
4. NewsAPI. (2024). *News API documentation*. <https://newsapi.org>
5. Fielding, R. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California.
6. Google Developers. (2024). *JWT: JSON Web Tokens explained*. <https://developers.google.com>
7. Winston.js. (2024). *Winston logging library documentation*. <https://github.com/winstonjs/winston>
8. Mozilla. (2024). *JavaScript basics*. <https://developer.mozilla.org>
9. Mozilla. (2024). *Fetch API documentation*. <https://developer.mozilla.org>
10. W3C. (2024). *HTML5 standards*. <https://www.w3.org>
11. W3C. (2024). *CSS3 standards*. <https://www.w3.org>
12. AWS. (2023). *REST API best practices*. <https://aws.amazon.com>
13. Google UX. (2023). *Principles of responsive design*. <https://design.google>
14. Microsoft. (2024). *Web security fundamentals*. <https://learn.microsoft.com>
15. Postman. (2024). *API testing guide*. <https://learning.postman.com>